

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Three

Oleh:

Putri Deissyta Herliyana

NIM. 2510817320010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Three ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Putri Deissyta Herliyana
NIM : 2510817320010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom,
Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI.....	iii
DAFTAR GAMBAR	iv
DAFTAR TABEL.....	Error! Bookmark not defined.
SOAL 1: Konversi Adjacency List ke Adjacency Matrix	6
A. Source Code	1
B. Output Program.....	3
C. Pembahasan.....	3
SOAL 2: Konversi Adjacency Matrix ke Adjacency List	6
A. Source Code	8
B. Output Program	10
C. Pembahasan	10
SOAL 3: BFS: Jarak Terpendek Keluar Labirin.....	12
A. Source Code	14
B. Output Program	16
C. Pembahasan	16
SOAL 4: DFS: Menghitung Semua Jalur Keluar Labirin.....	19
A. Source Code	21
B. Output Program	23
C. Pembahasan	23
SOAL 5: Tree Traversal.....	25
A. Source Code	27
B. Output Program	30
C. Pembahasan	30
Analisis Performa Keseluruhan	33
GITHUB.....	34

DAFTAR GAMBAR

Gambar 1. Output Program Konversi Adjacency List ke Adjacency Matrix	3
Gambar 2. Output Program Konversi Adjacency Matrix ke Adjacency List	10
Gambar 3. Output Program BFS: Jarak Terpendek Keluar Labirin.....	16
Gambar 4. Output Program DFS: Menghitung Semua Jalur Keluar Labirin.....	23
Gambar 5. Output Program Tree Traversal	30

DAFTAR TABEL

Tabel 1. Contoh Kasus Konversi Adjacency List ke Adjacency Matrix	1
Tabel 2. Source Code Konversi Adjacency List ke Adjacency Matrix	1
Tabel 3. Contoh Kasus Konversi Adjacency Matrix ke Adjacency List	7
Tabel 4. Source Code Konversi Adjacency Matrix ke Adjacency List	8
Tabel 5. Contoh Kasus BFS: Jarak Terpendek Keluar Labirin.....	14
Tabel 6. Source Code BFS: Jarak Terpendek Keluar Labirin.....	14
Tabel 7. Contoh Kasus DFS: Menghitung Semua Jalur Keluar Labirin.....	21
Tabel 8. Source Code DFS: Menghitung Semua Jalur Keluar Labirin.....	21
Tabel 9. Contoh Kasus Tree Traversal.....	26
Tabel 10. Source Code Tree Traversal.....	27

SOAL 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N M$

$U_1 V_1 W_1 U_2 V_2 W_2$

$\dots$

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .

- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

$V_1 V_2 \dots V_N$

$V_1 a_{11} a_{12} \dots a_{1N} V_2 a_{21} a_{22} \dots a_{2N}$

...

$V_N a_{N1} a_{N2} \dots a_{NN}$

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai

a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Tabel 1. Contoh Kasus Konversi Adjacency List ke Adjacency Matrix

Input	Output
5	Adjacency Matrix: A B C D E
A B C D E 6	A 0 4 2 0 0
A B 4	B 0 0 0 7 0
A C 2	C 0 1 0 0 6
B D 7	D 0 0 3 0 0
C B 1	E 0 0 0 0 0
C E 6	
D C 3	

A. Source Code

Tabel 2. Source Code Konversi Adjacency List ke Adjacency Matrix

1	#include <iostream>
2	#include <vector>
3	#include <string>
4	#include <map>
5	
6	using namespace std;
7	
8	int main() {
9	int N;
10	cin >> N;
11	
12	vector<string> labels(N);
13	map<string, int> index;
14	
15	for (int i = 0; i < N; i++) {
16	cin >> labels[i];
17	index[labels[i]] = i;
18	}


```

19
20     vector<vector<int>> matrix(N, vector<int>(N, 0));
21
22     int M;
23     cin >> M;
24
25     for (int i = 0; i < M; i++) {
26         string u, v;
27         int w;
28         cin >> u >> v >> w;
29         matrix[index[u]][index[v]] = w;
30     }
31
32     int colWidth = 4;
33
34     cout << "Adjacency Matrix:" << endl;
35
36     cout << " ";
37     for (int i = 0; i < N; i++) {
38         cout << " " << labels[i];
39     }
40     cout << endl;
41
42     for (int i = 0; i < N; i++) {
43         cout << labels[i];
44         for (int j = 0; j < N; j++) {
45             cout << " " << matrix[i][j];
46         }
47         cout << endl;
48     }

```

49	
50	return 0;
51	}

B. Output Program

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0

```

Gambar 1. Output Program Konversi Adjacency List ke Adjacency Matrix

C. Pembahasan

Kompleksitas Waktu

Pada proses konversi adjacency list ke adjacency matrix, program terlebih dahulu membuat matrix dengan ukuran $N \times N$ dan mengisi nilai awalnya dengan 0. Setelah itu, setiap edge yang terdapat pada adjacency list dibaca dan dimasukkan ke posisi yang sesuai berdasarkan vertex asal dan tujuan.

Proses pengisian edge dilakukan sebanyak jumlah edge yang ada, yaitu M kali. Maka kompleksitas waktu yang diperlukan adalah:

- **Best Case: $O(N^2)$**

Kondisi ini terjadi ketika jumlah edge sedikit, sehingga waktu program lebih banyak digunakan untuk membuat dan menginisialisasi matrix.

- **Average Case: $O(N^2 + M)$**

Program tetap harus membuat matrix terlebih dahulu, kemudian membaca seluruh edge yang tersedia.

- **Worst Case: $O(N^2)$**

Pada kondisi jumlah edge sangat banyak, jumlah edge dapat mencapai $N \times (N-1)$, tetapi pertumbuhannya masih berada pada tingkat $O(N^2)$.

Karakteristik Algoritma

Algoritma yang digunakan bersifat iteratif karena proses dilakukan menggunakan perulangan tanpa adanya pemanggilan fungsi secara rekursif. Setiap edge dibaca satu per satu, kemudian nilai bobotnya dimasukkan ke dalam matrix. Sebelum memasukkan edge, label vertex perlu diubah menjadi indeks agar dapat digunakan sebagai posisi pada matrix. Proses ini dibantu menggunakan map untuk menyimpan hubungan antara nama vertex dengan indeksnya.

Urutan data edge tidak memengaruhi hasil akhir karena setiap edge langsung ditempatkan sesuai posisi vertex asal dan tujuan.

Analisis Operasi

Berdasarkan hasil program, graph yang digunakan memiliki 5 vertex yaitu A, B, C, D, dan E dengan jumlah 6 edge. Setelah proses konversi, terbentuk adjacency matrix berukuran 5×5 .

Hasil pengisian matrix menunjukkan bahwa:

- Edge A menuju B dengan bobot 4 disimpan pada posisi baris A kolom B.
- Edge A menuju C dengan bobot 2 disimpan pada posisi baris A kolom C.
- Edge B menuju D dengan bobot 7 disimpan pada posisi baris B kolom D.
- Edge C menuju B dengan bobot 1 disimpan pada posisi baris C kolom B.
- Edge C menuju E dengan bobot 6 disimpan pada posisi baris C kolom E.

- Edge D menuju C dengan bobot 3 disimpan pada posisi baris D kolom C.

Vertex E tidak memiliki edge keluar sehingga seluruh nilai pada baris E bernilai 0.

Karena graph yang digunakan adalah graph berarah, hubungan A ke B tidak membuat hubungan B ke A secara otomatis.

SOAL 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN A11 A12 ... A1N A21 A22 ... A2N
...
AN1 AN2 ... ANN
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .

- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

$V_1 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

$V_2 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

...

$V_N \rightarrow \dots$

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol $\rightarrow$.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Tabel 3. Contoh Kasus Konversi Adjacency Matrix ke Adjacency List

Input	Output
5	Adjacency List:

A B C D E	A -> (B,4) , (C,2)
0 4 2 0 0	B -> (D,7)
0 0 0 7 0	C -> (B,1) , (E,6)
0 1 0 0 6	D -> (C,3)
0 0 3 0 0	E -> -
0 0 0 0 0	

A. Source Code

Tabel 4. Source Code Konversi Adjacency Matrix ke Adjacency List

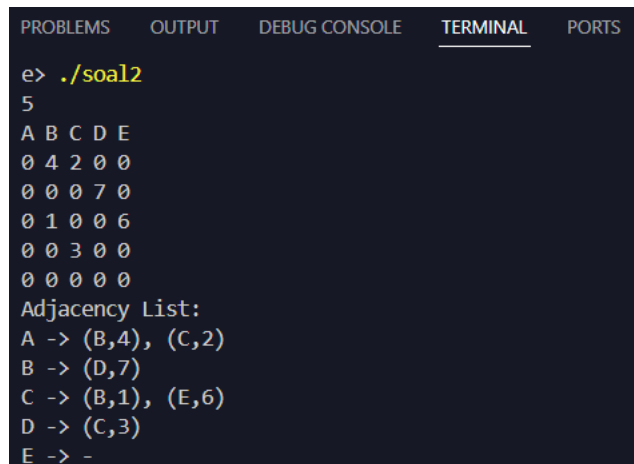
1	#include <iostream>
2	#include <vector>
3	#include <string>
4	
5	using namespace std;
6	
7	int main() {
8	int N;
9	cin >> N;
10	
11	vector<string> labels(N);
12	for (int i = 0; i < N; i++) {
13	cin >> labels[i];
14	}
15	
16	vector<vector<int>> matrix(N, vector<int>(N));
17	for (int i = 0; i < N; i++) {
18	for (int j = 0; j < N; j++) {
19	cin >> matrix[i][j];
20	}

```

21     }
22
23     cout << "Adjacency List:" << endl;
24
25     for (int i = 0; i < N; i++) {
26         cout << labels[i] << " -> ";
27
28         bool hasEdge = false;
29         bool first = true;
30
31         for (int j = 0; j < N; j++) {
32             if (matrix[i][j] > 0) {
33                 hasEdge = true;
34                 if (!first) cout << ", ";
35                 cout << "(" << labels[j] << "," <<
36 matrix[i][j] << ")";
37                 first = false;
38             }
39         }
40
41         if (!hasEdge) {
42             cout << "-";
43         }
44
45         cout << endl;
46     }
47
48     return 0;
49 }

```


B. Output Program



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

e> ./soal2
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
```

Gambar 2. Output Program Konversi Adjacency Matrix ke Adjacency List

C. Pembahasan

Kompleksitas Waktu

Konversi adjacency matrix ke adjacency list dilakukan dengan memeriksa seluruh isi matrix. Program membaca setiap baris sebagai vertex asal, lalu mengecek setiap kolom untuk melihat apakah terdapat edge menuju vertex lain.

Karena semua elemen matrix harus diperiksa, maka jumlah operasi bergantung pada ukuran matrix yaitu $N \times N$.

Kompleksitas waktunya Adalah:

- Best Case: $O(N^2)$
Walaupun matrix kosong atau tidak memiliki edge, program tetap harus mengecek seluruh isi matrix.
- Average Case: $O(N^2)$
Pada kondisi umum, seluruh elemen tetap diperiksa untuk menemukan edge yang tersedia.
- Worst Case: $O(N^2)$

Jumlah edge yang banyak tidak mengurangi proses pengecekan karena semua bagian matrix tetap dibaca.

Karakteristik Algoritma

Algoritma ini menggunakan perulangan bersarang. Perulangan pertama digunakan untuk membaca setiap vertex, sedangkan perulangan kedua digunakan untuk mengecek seluruh kemungkinan tujuan vertex tersebut. Jika ditemukan nilai lebih dari 0 pada matrix, berarti terdapat edge dari vertex baris menuju vertex kolom. Data tersebut kemudian dimasukkan ke adjacency list. Algoritma ini tidak membutuhkan data yang sudah terurut karena proses hanya mengikuti posisi data pada matrix.

Analisis Operasi

Dari hasil program dengan input matrix berukuran 5×5 , adjacency matrix berhasil dikembalikan menjadi adjacency list.

Hasil konversinya adalah:

- A memiliki edge menuju B dengan bobot 4 dan menuju C dengan bobot 2.
- B memiliki edge menuju D dengan bobot 7.
- C memiliki edge menuju B dengan bobot 1 dan menuju E dengan bobot 6.
- D memiliki edge menuju C dengan bobot 3.
- E tidak memiliki edge keluar sehingga ditampilkan dengan tanda "-".

Dari total 25 elemen matrix yang diperiksa, terdapat 6 edge yang berhasil ditemukan. Tanda "-" menunjukkan bahwa vertex tersebut tidak memiliki hubungan keluar.

SOAL 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C} G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC} SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.

- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Tabel 5. Contoh Kasus BFS: Jarak Terpendek Keluar Labirin

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (7,9). Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel (2,1).

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari

(0,0) ke (7,9) adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 6. Source Code BFS: Jarak Terpendek Keluar Labirin

1	#include <iostream>
2	#include <vector>
3	#include <queue>
4	
5	using namespace std;
6	
7	int main() {
8	int R, C;

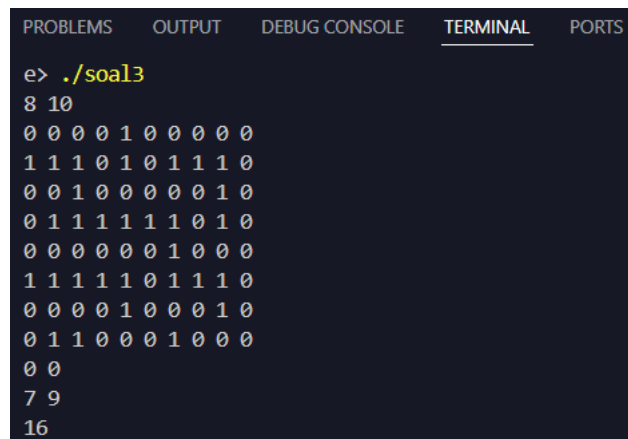
9	cin >> R >> C;
10	
11	vector<vector<int>> grid(R, vector<int>(C));
12	for (int i = 0; i < R; i++)
13	for (int j = 0; j < C; j++)
14	cin >> grid[i][j];
15	
16	int sr, sc, fr, fc;
17	cin >> sr >> sc >> fr >> fc;
18	
19	vector<vector<int>> dist(R, vector<int>(C, -1));
20	queue<pair<int,int>> q;
21	
22	dist[sr][sc] = 0;
23	q.push({sr, sc});
24	
25	int dr[] = {-1, 1, 0, 0};
26	int dc[] = {0, 0, -1, 1};
27	
28	while (!q.empty()) {
29	int row = q.front().first;
30	int col = q.front().second;
31	q.pop();
32	
33	for (int d = 0; d < 4; d++) {
34	int nr = row + dr[d];
35	int nc = col + dc[d];
36	
37	if (nr >= 0 && nr < R && nc >= 0 && nc <
38	C

```

39         && grid[nr][nc] == 0 && dist[nr][nc]
40 == -1) {
41             dist[nr][nc] = dist[row][col] + 1;
42             q.push({nr, nc});
43         }
44     }
45 }
46
47 cout << dist[fr][fc] << endl;
48
49 return 0;
50 }

```

B. Output Program



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
e> ./soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16

```

Gambar 3. Output Program BFS: Jarak Terpendek Keluar Labirin

C. Pembahasan

Kompleksitas Waktu

Algoritma BFS (Breadth First Search) digunakan untuk mencari jarak terpendek dari posisi awal menuju posisi tujuan pada labirin. Cara kerja BFS adalah

dengan mengecek posisi yang memiliki jarak paling dekat terlebih dahulu, kemudian melanjutkan ke posisi berikutnya.

Setiap kotak pada labirin hanya diproses satu kali karena program menyimpan informasi jarak pada array dist. Jika sebuah kotak sudah pernah dikunjungi, maka kotak tersebut tidak akan diproses ulang.

Kompleksitas waktu yang diperoleh yaitu:

- Best Case: $O(1)$
Kondisi ini terjadi apabila posisi awal sama dengan posisi tujuan, sehingga program tidak perlu melakukan pencarian.
- Average Case: $O(RxC)$
Pada kondisi umum, program hanya perlu mengunjungi sebagian area labirin sebelum menemukan tujuan.
- Worst Case: $O(RxC)$
Kondisi ini terjadi ketika hampir seluruh bagian labirin harus diperiksa sebelum tujuan ditemukan atau dinyatakan tidak dapat dicapai.

Karakteristik Algoritma

BFS menggunakan struktur data queue (antrian) untuk menyimpan posisi yang akan diperiksa. Posisi yang masuk lebih awal akan diproses lebih dulu sehingga pencarian berjalan berdasarkan tingkat jarak dari titik awal.

Algoritma ini bersifat iteratif karena tidak menggunakan rekursi. Program mengambil posisi dari queue, kemudian mengecek empat arah pergerakan yaitu atas, bawah, kiri, dan kanan.

Jika posisi baru valid dan belum pernah dikunjungi, posisi tersebut dimasukkan ke queue dan jaraknya ditambah satu.

Analisis Operasi

Berdasarkan hasil pengujian, labirin memiliki ukuran 8×10 dengan titik awal berada di (0,0) dan titik tujuan berada di (7,9). Program menghasilkan nilai 16, yang berarti jarak minimum dari awal menuju tujuan adalah 16 langkah.

BFS dapat menemukan jalur terpendek karena pencarian dilakukan berdasarkan jarak. Posisi yang lebih dekat akan selalu diperiksa terlebih dahulu dibandingkan posisi yang lebih jauh.

Proses pencarian dapat dijelaskan sebagai berikut:

- Titik awal memiliki jarak 0.
- Setiap perpindahan ke kotak berikutnya menambah jarak sebesar 1.
- Ketika titik tujuan ditemukan, nilai jarak yang diperoleh merupakan jarak minimum.

Kotak yang sudah memiliki nilai jarak tidak akan diproses kembali sehingga pencarian menjadi lebih efisien.

SOAL 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C G21 G22 ... G2C
...
GR1 GR2 ... GRC SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .

- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Tabel 7. Contoh Kasus DFS: Menghitung Semua Jalur Keluar Labirin

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

A. Source Code

Tabel 8. Source Code DFS: Menghitung Semua Jalur Keluar Labirin

1	#include <iostream>
2	#include <vector>
3	
4	using namespace std;
5	
6	int R, C;
7	vector<vector<int>> grid;
8	vector<vector<bool>> visited;
9	int fr, fc;
10	int totalPaths;
11	
12	int dr[] = {-1, 1, 0, 0};
13	int dc[] = {0, 0, -1, 1};
14	
15	void dfs(int row, int col) {

```

16     if (row == fr && col == fc) {
17         totalPaths++;
18         return;
19     }
20
21     for (int d = 0; d < 4; d++) {
22         int nr = row + dr[d];
23         int nc = col + dc[d];
24
25         if (nr >= 0 && nr < R && nc >= 0 && nc < C
26             && grid[nr][nc] == 0 && !visited[nr][nc])
27     {
28         visited[nr][nc] = true;
29         dfs(nr, nc);
30         visited[nr][nc] = false; // backtrack
31     }
32 }
33 }
34
35 int main() {
36     cin >> R >> C;
37
38     grid.assign(R, vector<int>(C));
39     for (int i = 0; i < R; i++)
40         for (int j = 0; j < C; j++)
41             cin >> grid[i][j];
42
43     int sr, sc;
44     cin >> sr >> sc >> fr >> fc;
45

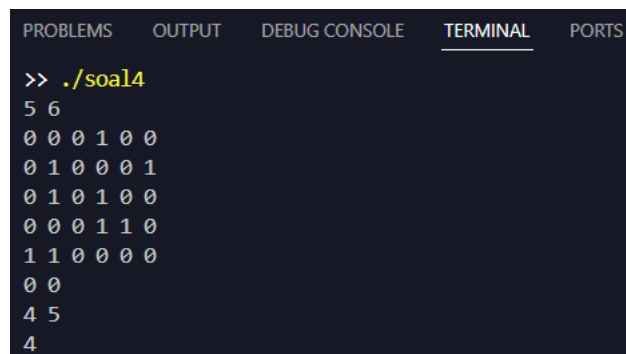
```

```

46     visited.assign(R, vector<bool>(C, false));
47     totalPaths = 0;
48
49     visited[sr][sc] = true;
50     dfs(sr, sc);
51
52     cout << totalPaths << endl;
53
54     return 0;
55 }

```

B. Output Program



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
>> ./soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

```

Gambar 4. Output Program DFS: Menghitung Semua Jalur Keluar Labirin

C. Pembahasan

Kompleksitas Waktu

DFS (Depth First Search) digunakan untuk mencari seluruh kemungkinan jalur dari titik awal menuju titik tujuan. Berbeda dengan BFS yang hanya mencari jarak terpendek, DFS akan mencoba setiap kemungkinan jalan yang tersedia.

Kompleksitas waktu algoritma ini adalah:

- Best Case $O(R \times C)$
Terjadi apabila jalur menuju tujuan langsung ditemukan tanpa banyak percabangan.
- Average Case: bergantung pada kondisi labirin

Banyaknya percabangan dan jalan yang tersedia memengaruhi jumlah pencarian yang dilakukan.

- Worst Case: $O((R \times C)!)$

Pada kondisi terburuk, program dapat mencoba banyak kemungkinan jalur sebelum mendapatkan hasil akhir.

Karakteristik Algoritma

DFS pada program ini menggunakan metode rekursif dengan konsep backtracking. Program akan berjalan terus menuju satu arah sampai menemukan tujuan atau jalan buntu.

Setiap kali masuk ke sebuah kotak, kotak tersebut diberi tanda sudah dikunjungi agar tidak dilewati kembali dalam jalur yang sama. Jika ternyata jalur tersebut tidak menghasilkan solusi, program akan kembali ke posisi sebelumnya dan mencoba arah lain.

Data tidak perlu dalam kondisi terurut karena DFS hanya mengikuti jalur yang tersedia pada labirin.

Analisis Operasi

Berdasarkan hasil program, labirin berukuran 5×6 dengan posisi awal (0,0) dan tujuan (4,5). Hasil output menunjukkan terdapat 4 jalur berbeda yang dapat digunakan untuk mencapai tujuan.

Proses DFS dilakukan dengan mencoba setiap kemungkinan jalan. Jika sebuah jalur tidak bisa mencapai tujuan, program melakukan backtracking dan kembali mencoba jalur lainnya.

Beberapa jalur yang menuju jalan buntu tetap diperiksa oleh DFS, tetapi tidak dihitung sebagai solusi karena tidak sampai ke titik akhir. Hasil akhir menunjukkan bahwa terdapat 4 jalur valid dari posisi awal menuju posisi tujuan.

SOAL 5: Tree Traversal

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red-Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N Q$

$T_1 T_2$

$\dots T_Q$

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .

- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-1000000000 \leq A_i \leq 1000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Tabel 9. Contoh Kasus Tree Traversal

Input	Output
7	[Preorder] : 50 30 20 40 70
50 30 70 20 40 60 80	60 80
4	[Inorder] : 20 30 40 50 60
PREORDER INORDER POSTORDER	70 80
ALL	[Postorder] : 20 40 30 60 80
	70 50

	[Preorder] : 50 30 20 40 70 60 80 [Inorder] : 20 30 40 50 60 70 80 [Postorder] : 20 40 30 60 80 70 50
--	--

A. Source Code

Tabel 10. Source Code Tree Traversal

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include "RedBlackTree.h"
5
6  using namespace std;
7
8  void preorder(const RedBlackTree::Node* node, const
9  RedBlackTree::Node* nil,
10               vector<int>& result) {
11      if (node == nil) return;
12      result.push_back(node->key);
13      preorder(node->left, nil, result);
14      preorder(node->right, nil, result);
15  }
16
17  void inorder(const RedBlackTree::Node* node, const
18  RedBlackTree::Node* nil,
19               vector<int>& result) {
20      if (node == nil) return;

```

```

21     inorder(node->left, nil, result);
22     result.push_back(node->key);
23     inorder(node->right, nil, result);
24 }
25
26 void postorder(const RedBlackTree::Node* node, const
27 RedBlackTree::Node* nil,
28     vector<int>& result) {
29     if (node == nil) return;
30     postorder(node->left, nil, result);
31     postorder(node->right, nil, result);
32     result.push_back(node->key);
33 }
34
35 void printTraversal(const string& label, const
36 vector<int>& result) {
37     cout << label << " : ";
38     for (int i = 0; i < (int)result.size(); i++) {
39         if (i > 0) cout << " ";
40         cout << result[i];
41     }
42     cout << endl;
43 }
44
45 int main() {
46     int N;
47     cin >> N;
48
49     RedBlackTree rbt;
50

```

```

51     for (int i = 0; i < N; i++) {
52         int val;
53         cin >> val;
54         if (!rbt.contains(val)) {
55             rbt.insert(val);
56         }
57     }
58
59     int Q;
60     cin >> Q;
61
62     while (Q--) {
63         string query;
64         cin >> query;
65
66         if (rbt.empty()) {
67             cout << "Tree kosong. Tidak ada yang bisa
68 ditampilkan." << endl;
69             continue;
70         }
71
72         vector<int> pre, in, post;
73         preorder(rbt.root(), rbt.nil(), pre);
74         inorder(rbt.root(), rbt.nil(), in);
75         postorder(rbt.root(), rbt.nil(), post);
76
77         if (query == "PREORDER") {
78             printTraversal("[Preorder]", pre);
79         } else if (query == "INORDER") {
80             printTraversal("[Inorder]", in);

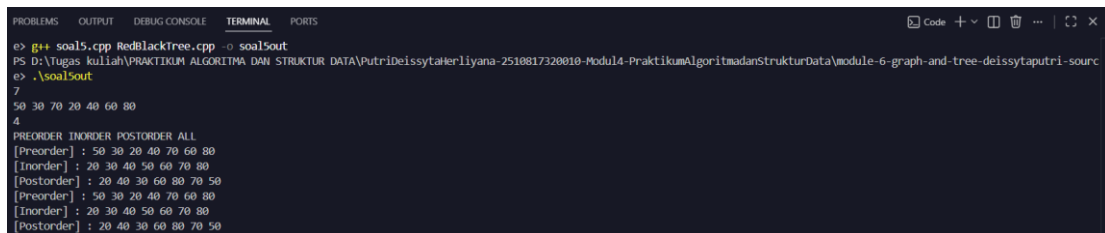
```

```

81         } else if (query == "POSTORDER") {
82             printTraversal("[Postorder]", post);
83         } else if (query == "ALL") {
84             printTraversal("[Preorder]", pre);
85             printTraversal("[Inorder]", in);
86             printTraversal("[Postorder]", post);
87         }
88     }
89
90     return 0;
91 }

```

B. Output Program



```

e> g++ soal5.cpp RedBlackTree.cpp -o soal5out
PS D:\Tugas kuliah\PRAKTIKUM ALGORITMA DAN STRUKTUR DATA\PutriDeissyatierliyana-2510817320010-Modul4-PraktikumAlgoritmadanStrukturData\module-6-graph-and-tree-deissyaputri-sourc
e> .\soal5out
7
50 30 70 20 40 60 80
4
PREORDER INORDER POSTORDER ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50

```

Gambar 5. Output Program Tree Traversal

C. Pembahasan

Kompleksitas Waktu

Red-Black Tree merupakan salah satu jenis Binary Search Tree yang menjaga keseimbangan pohon melalui aturan warna dan proses rotasi. Dengan struktur yang seimbang, proses pencarian dan penyisipan data dapat dilakukan dengan waktu yang relatif cepat.

Operasi insert pada Red-Black Tree memiliki kompleksitas:

$O(\log n)$

Hal ini dikarenakan tinggi pohon tetap terjaga sehingga pencarian posisi data baru tidak perlu melewati seluruh node. Sebelum melakukan insert, program melakukan pengecekan apakah data sudah ada atau belum. Proses pengecekan tersebut juga memiliki kompleksitas $O(\log n)$

Untuk proses traversal, setiap node hanya dikunjungi satu kali sehingga kompleksitasnya:

- **Best Case: $O(n)$**
- **Average Case: $O(n)$**
- **Worst Case: $O(n)$**

Karakteristik Algoritma

Traversal pada Red-Black Tree menggunakan metode rekursif. Terdapat tiga jenis traversal yang digunakan yaitu preorder, inorder, dan postorder.

- Preorder mengunjungi root terlebih dahulu, kemudian subtree kiri dan kanan.
- Inorder mengunjungi subtree kiri, root, lalu subtree kanan.
- Postorder mengunjungi subtree kiri dan kanan terlebih dahulu, kemudian root.

Karena Red-Black Tree tetap mengikuti aturan Binary Search Tree, hasil inorder selalu menghasilkan data dalam urutan dari nilai terkecil ke terbesar.

Analisis Operasi

Pada pengujian program, terdapat 7 data yang dimasukkan yaitu 50, 30, 70, 20, 40, 60, dan 80. Semua data berhasil dimasukkan karena tidak terdapat nilai yang sama.

Hasil traversal yang diperoleh:

- Preorder:

50 30 20 40 70 60 80

Pada traversal ini, node utama yaitu 50 dicetak terlebih dahulu, kemudian dilanjutkan ke bagian kiri dan kanan tree.

- Inorder:

20 30 40 50 60 70 80

Hasil ini menunjukkan bahwa struktur Binary Search Tree berjalan dengan benar karena data tampil secara berurutan.

- Postorder:

20 40 30 60 80 70 50

Pada traversal ini, node root dicetak terakhir setelah seluruh bagian tree selesai dikunjungi.

Meskipun Red-Black Tree melakukan rotasi dan perubahan warna untuk menjaga keseimbangan, aturan Binary Search Tree tetap dipertahankan sehingga hasil traversal tetap sesuai.

Analisis Performa Keseluruhan

Berdasarkan hasil percobaan, setiap algoritma memiliki fungsi dan penggunaan yang berbeda. Pada bagian graph, konversi adjacency list ke adjacency matrix dan sebaliknya memiliki kompleksitas $O(N^2)$ karena data perlu diproses berdasarkan jumlah vertex. Adjacency matrix lebih cepat untuk mengecek hubungan antar vertex, sedangkan adjacency list lebih hemat penyimpanan karena hanya menyimpan edge yang tersedia.

Pada pencarian labirin, BFS dan DFS memiliki perbedaan tujuan. BFS digunakan untuk mencari jalur terpendek karena pencarian dilakukan berdasarkan jarak. Hasil percobaan BFS menunjukkan jarak minimum dari start ke finish adalah 16 langkah. Sementara itu, DFS digunakan untuk mencari seluruh kemungkinan jalur dan pada percobaan berhasil menemukan 4 jalur yang valid.

Pada struktur tree, Red-Black Tree menjaga keseimbangan pohon sehingga proses insert dan pencarian dapat berjalan dengan kompleksitas $O(\log n)$. Hasil inorder yang menghasilkan urutan 20 30 40 50 60 70 80 menunjukkan bahwa struktur tree sudah sesuai dengan aturan Binary Search Tree.

Dari seluruh percobaan dapat disimpulkan bahwa pemilihan algoritma harus disesuaikan dengan kebutuhan. BFS cocok untuk mencari jalur terpendek, DFS untuk mengecek semua kemungkinan jalur, adjacency matrix/list digunakan sesuai karakteristik graph, dan Red-Black Tree digunakan untuk menjaga performa pencarian data.

GITHUB

<https://github.com/DSA26-ULM/module-6-graph-and-tree-deissytaputri-source>